

Examen Grupal

MyCobot 280

Cinemática, Control y Visión de Robots

ROB – Curso 2026

Duración: 1 semana

4 Roles – Uso obligatorio del API pymycobot

Robot: Yahboom MyCobot 280 (6-DOF serial)

Puerto: `/dev/ttyUSB0` – Baud rate: 1 000 000

Entorno: Ubuntu 20.04 + ROS 2 Foxy + Jetson Nano

Contents

1	Descripción General	2
2	Problemas que el Equipo debe Resolver	3
2.1	Tabla resumen de problemas	3
2.2	Sub-problemas detallados	3
3	Marco Teórico – Cinemática del MyCobot 280	6
3.1	Parámetros DH	6
3.2	Cinemática Directa (P2)	6
3.3	Cinemática Inversa (P3)	7
3.4	Evasión de Colisiones – Problema Abierto (P4)	7
4	Roles del Equipo	9
4.1	Matriz de participación por competencia	9
4.2	Rol 1: Líder de Integración	9
4.3	Rol 2: Especialista en Cinemática	10
4.4	Rol 3: Ingeniero de Control	11
4.5	Rol 4: Ingeniero de Visión	11
5	Cronograma Semanal	13
6	Entregables	14
7	Rúbrica de Evaluación	15
8	Referencia Rápida del API pymycobot	16
9	Normas de Seguridad	17

1. Descripción General

El presente examen evalúa la capacidad del equipo para analizar, programar y controlar un brazo robótico serial de 6 grados de libertad (MyCobot 280). El trabajo integra cinemática directa e inversa, planificación de trayectorias, detección de objetos por visión computacional y control real del robot mediante el API `pymycobot`.

Principio Fundamental

Todos los integrantes deben conocer y poder explicar **todas las partes del sistema**. Los roles definen quién tiene **mayor responsabilidad y profundidad** en cada área, no quién es el único que la conoce. **Cualquier integrante puede ser preguntado sobre cualquier componente** durante la defensa oral.

Cada integrante debe realizar llamadas directas al API del robot en sus entregables.

No es válido reutilizar código de otro integrante sin haberlo ejecutado y verificado personalmente con el hardware real.

2. Problemas que el Equipo debe Resolver

El examen está estructurado alrededor de **7 problemas concretos**. Cada rol lidera la solución de los problemas de su área, pero todos los integrantes participan en la comprensión y verificación de cada solución.

2.1 Tabla resumen de problemas

#	Problema	Descripción	Dificultad
P1	Representación DH	Establecer los parámetros DH y construir las matrices de transformación homogénea.	Alta
P2	Cinemática directa	Dado $[\theta_1, \dots, \theta_6]$, calcular la posición del extremo. Verificar con <code>get_coords()</code> .	Alta
P3	Cinemática inversa	Dada $[x, y, z]$, calcular los ángulos necesarios. Verificar con <code>get_angles()</code> .	Alta
P4	Evasión de colisiones	Identificar configs peligrosas. Proponer e implementar estrategia de evasión (problema abierto).	Alta
P5	Control de trayectorias	Implementar el ciclo de agarre. Ejecutar 5 ciclos consecutivos sin intervención humana.	Media
P6	Detección de objetos	Detectar (c_x, c_y) en píxeles y convertirla a coordenadas del robot en mm.	Media
P7	Pipeline <i>end-to-end</i>	Integrar visión, cinemática y control: detectar $\rightarrow$ IK $\rightarrow$ agarrar $\rightarrow$ depositar.	Alta

2.2 Sub-problemas detallados

P1 – Representación cinemática (DH)

1. Identificar los 6 sistemas de referencia según la convención Denavit-Hartenberg.
2. Construir la tabla de parámetros: a_i , d_i , α_i , θ_i y rango por articulación.
3. Derivar la matriz de transformación homogénea T_i para cada *joint*.
4. Verificar que T_0^6 en la pose *home* $[0, 0, 0, 0, 0, 0]$ reproduce la posición de referencia del robot.
5. Documentar el significado físico de cada parámetro en el contexto del MyCobot 280.

P2 – Cinemática directa (FK)

1. Implementar $T_0^6 = T_1 \cdot T_2 \cdot T_3 \cdot T_4 \cdot T_5 \cdot T_6$ en Python con `numpy`.
2. Calcular la posición del extremo para al menos 5 configuraciones de *joints* distintas.

3. Comparar la posición calculada con `mc.get_coords()` y tabular el error en mm.
4. Identificar y analizar las fuentes de discrepancia entre el modelo DH y el robot físico.
5. Graficar el espacio de trabajo alcanzable (nube de puntos 2D en el plano XZ).

P3 – Cinemática inversa (IK)

1. Implementar IK vía API: `mc.send_coords([x,y,z,rx,ry,rz], 30, 1)` y leer `mc.get_angles()`.
2. Implementar IK analítica simplificada: $\theta_1 = \text{atan2}(y, x)$; modelo planar 2R para J_2 - J_3 .
3. Comparar IK analítica vs. solución API para al menos 3 posiciones cartesianas distintas.
4. Tabular el error en grados entre ambas soluciones y discutir las causas.
5. Identificar configuraciones singulares y posiciones fuera del espacio de trabajo alcanzable.

P4 – Evasión de colisiones (problema abierto)

1. Identificar al menos 2 tipos de colisión potencial en el escenario real del laboratorio.
2. Proponer una estrategia de evasión justificada y apropiada para el MyCobot 280.
3. Implementar el mecanismo elegido e integrarlo en el flujo de movimiento del robot.
4. Validar con el robot real que no ocurren colisiones durante la ejecución del pipeline.
5. Documentar el método, sus limitaciones y los criterios de validación (mínimo 1 página).

P5 – Control de trayectorias

1. Definir las poses clave del ciclo: `init_pose`, `watch_pose`, `pick_pose` y `place_pose`.
2. Implementar el ciclo de agarre con `send_angles/send_coords + set_gripper_value`.
3. Calibrar velocidades y tiempos de espera para movimientos seguros y repetibles.
4. Ejecutar 5 ciclos consecutivos sin intervención humana y registrar la tasa de éxito.
5. Gestionar errores de comunicación con el robot (reintentos, *timeouts*).

P6 – Detección de objetos

1. Calibrar los rangos HSV para el objeto objetivo en las condiciones del laboratorio.
2. Detectar el contorno y calcular el centroide (c_x, c_y) en píxeles con OpenCV.
3. Implementar la transformación píxel $\rightarrow$ mm en el sistema de referencia del robot.
4. Validar la precisión de detección para al menos 3 posiciones distintas del objeto.
5. Manejar falsos positivos, iluminación variable y ausencia del objeto en la escena.

P7 – Pipeline *end-to-end*

1. Diseñar la máquina de estados: IDLE → DETECTAR → CALC_IK → AGARRAR → DEPOSITAR.
2. Integrar los módulos de visión, cinemática y control en `main.py`.
3. Implementar manejo de errores para cada estado de la máquina.
4. Ejecutar al menos 5 ciclos autónomos completos (detección + IK + agarre + depósito).
5. Grabar vídeo de demostración y documentar fallos encontrados y cómo se resolvieron.

Distribución de liderazgo por rol

- **Líder de Integración:** P7 (pipeline E2E)
- **Especialista en Cinemática:** P1, P2, P3, P4
- **Ingeniero de Control:** P5
- **Ingeniero de Visión:** P6

Todos los integrantes deben poder explicar cualquiera de los 7 problemas durante la defensa oral.

3. Marco Teórico – Cinemática del MyCobot 280

El MyCobot 280 es un brazo serial de 6 grados de libertad. El equipo usa la convención Denavit-Hartenberg (DH) para su análisis cinemático.

3.1 Parámetros DH

Articulación	a_i (mm)	d_i (mm)	α_i	θ_i	Rango (°)
J_1 Base	0	131.56	90°	θ_1	−168 a 168
J_2 Hombro	110.4	0	0°	θ_2	−135 a 90
J_3 Codo	96	0	0°	θ_3	−150 a 150
J_4 Muñeca 1	0	66.39	-90°	θ_4	−145 a 145
J_5 Muñeca 2	0	73.18	90°	θ_5	−165 a 165
J_6 Gripper	0	48.6	0°	θ_6	−180 a 180

Nota: a_i y d_i en mm. θ_i es la variable articular. Alcance máximo extendido: ≈ 280 mm.

3.2 Cinemática Directa (P2)

La matriz de transformación DH para el eslabón i es:

$$T_i(\theta_i) = R_z(\theta_i) T_z(d_i) T_x(a_i) R_x(\alpha_i) \quad (1)$$

Desarrollada explícitamente:

$$T_i = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2)$$

La cinemática directa completa del extremo:

$$T_0^6 = T_1 \cdot T_2 \cdot T_3 \cdot T_4 \cdot T_5 \cdot T_6 \quad (3)$$

La posición del extremo (x_e, y_e, z_e) corresponde a la **última columna** (filas 1–3) de T_0^6 .

Verificación FK con el API

```
mc.send_angles([j1, j2, j3, j4, j5, j6], 30)
time.sleep(2)
pos_real = mc.get_coords() # [x, y, z, rx, ry, rz]
# Comparar con el cálculo manual de FK
```

3.3 Cinemática Inversa (P3)

Enfoque 1 – IK numérica vía API (todos los roles)

$$\text{mc.send_coords}([x, y, z, rx, ry, rz], 30, 1) \quad (4)$$

El API resuelve la IK internamente y mueve el robot. Los ángulos resultantes se leen con `mc.get_angles()`.

Enfoque 2 – IK analítica simplificada (Rol Cinemática)

Para los primeros tres *joints* (modelo planar 2R + base):

$$\theta_1 = \text{atan2}(y, x) \quad (5)$$

$$r = \sqrt{x^2 + y^2}, \quad z' = z - d_1 \quad (6)$$

$$\cos \theta_3 = \frac{r^2 + z'^2 - L_2^2 - L_3^2}{2 L_2 L_3} \quad (7)$$

$$\theta_3 = \text{atan2}\left(\pm \sqrt{1 - \cos^2 \theta_3}, \cos \theta_3\right) \quad (8)$$

$$\theta_2 = \text{atan2}(z', r) - \text{atan2}(L_3 \sin \theta_3, L_2 + L_3 \cos \theta_3) \quad (9)$$

donde $L_2 = 110.4 \text{ mm}$, $L_3 = 96 \text{ mm}$ y $d_1 = 131.56 \text{ mm}$.

3.4 Evasión de Colisiones – Problema Abierto (P4)

El MyCobot 280 puede colisionar con:

- (a) **Sí mismo:** configuraciones de *joints* que causan auto-colisión de los eslabones.
- (b) **La mesa de trabajo:** el extremo o los eslabones impactan la superficie.
- (c) **Objetos del entorno:** zonas de depósito, cámara u otros obstáculos fijos.

Enfoques posibles (el equipo elige uno o combina varios):

1. **Límites conservadores:** definir rangos más estrechos que los físicos y verificar antes de `send_angles()`.
2. **Posición intermedia segura:** en lugar de ir directamente al destino, pasar por un punto con *clearance* garantizado.
3. **Verificación de altura mínima:** calcular Z del extremo vía FK antes de ejecutar; rechazar si $Z < Z_{\text{seguro}}$.
4. **Detección geométrica simplificada:** modelar eslabones como segmentos y verificar no intersección con planos de obstáculo.
5. **Planificación reactiva:** si `mc.get_coords()` retorna $Z < Z_{\text{umbral}}$, ejecutar pose de emergencia pre-planificada.

No se evalúa la complejidad del método elegido, sino:

1. Identificación correcta de al menos 2 escenarios de colisión en el laboratorio real.
2. Justificación del enfoque elegido (por qué es apropiado para este robot y escenario).
3. Implementación funcional que demuestre que el robot no colisiona durante la ejecución.
4. Documentación del método en el informe técnico (mínimo 1 página).

Un método simple pero bien justificado e implementado vale más que uno complejo sin validación.

4. Roles del Equipo

Los 4 roles son complementarios. Todos los integrantes conocen el sistema completo; la diferencia es el nivel de profundidad y responsabilidad en cada área.

4.1 Matriz de participación por competencia

Competencia	Líder	Cinemática	Control	Visión
Cinemática FK/IK	Conoce	Lidera	Aplica	Conoce
Control API robot	Coordina	Aplica	Lidera	Usa
Evasión colisiones	Coordina	Lidera	Aplica	Conoce
Trayectorias/agarre	Coordina	Calcula IK	Lidera	Conoce
Visión HSV/cámara	Coordina	Conoce	Usa	Lidera
Integración sistema	Lidera	Colabora	Colabora	Colabora
Informe técnico	Lidera	Secc. CIN	Secc. CTRL	Secc. VIS

*Leyenda: **Lidera** = responsable principal / **Aplica** = implementa activamente / Coordina = supervisa / Conoce = comprende y puede explicar / Usa = consume el resultado.*

4.2 Rol 1: Líder de Integración

[LÍDER] Líder de Integración

Problemas que lidera: **P7** (pipeline E2E)

Tareas:

- Definir la arquitectura del sistema y el flujo de datos entre los 3 módulos.
- Crear el repositorio Git con estructura: `/cinematica`, `/control`, `/vision`, `/main`.
- Implementar `main.py`: detectar (visión) → calcular IK (cinemática) → agarrar (control).
- Máquina de estados: `IDLE` → `DETECTANDO` → `CALC_IK` → `AGARRANDO` → `DEPOSITAR`.
- Logging centralizado con *timestamp* para todas las operaciones del sistema.
- Verificar que cada integrante demuestre llamadas directas al API del MyCobot.
- Preparar la presentación y coordinar el vídeo de demostración.

Entregables:

- `main.py` funcional integrando los 3 módulos y resolviendo P7.
- Diagrama de arquitectura del sistema.
- Repositorio Git con historial de *commits* de todos los integrantes.

- Log de al menos 1 sesión completa (mínimo 5 ciclos).
- Diapositivas de presentación + vídeo de 3 minutos.

API requerida:

```
mc = MyCobot("/dev/ttyUSB0", 1000000)
mc.send_angles([0, 0, 0, 0, 0, -45], 50) # init_pose
mc.get_angles() # verificar estado
mc.is_controller_connected() # check al startup
```

4.3 Rol 2: Especialista en Cinemática

[CINEMÁTICA] Especialista en Cinemática (FK + IK + Colisiones)

Problemas que lidera: P1, P2, P3, P4

Tareas:

- P1: Construir los parámetros DH del MyCobot 280 y las 6 matrices de transformación.
- P2: Implementar FK completa en Python/numpy. Calcular posición para 5 configuraciones distintas.
- P2: Verificar FK calculada vs. `mc.get_coords()` – tabular error en mm.
- P3: Implementar IK analítica simplificada (primeros 3 *joints*, modelo planar 2R + base).
- P3: Verificar IK calculada vs. `mc.get_angles()` tras `mc.send_coords()` – tabular error en grados.
- P4: Identificar 2+ escenarios de colisión y elegir un enfoque de evasión.
- P4: Implementar y validar el mecanismo de evasión con el robot real.
- Graficar el espacio de trabajo alcanzable (nube de puntos 2D, plano XZ, con matplotlib).

Entregables:

- `cinematica.py` con clases `ForwardKinematics`, `InverseKinematics` y `CollisionChecker`.
- Tabla FK: 5 configuraciones – calculada vs. real, con error en mm.
- Tabla IK: 3 posiciones cartesianas – calculada vs. real, con error en grados.
- Función `ik_solve(x,y,z)` utilizada por el Rol de Control.
- Gráfica del espacio de trabajo en plano XZ.
- Reporte de cinemática (3 págs.): DH, FK, IK, evasión de colisiones y justificación.

API requerida:

```
mc.send_angles([j1,j2,j3,j4,j5,j6], 30) # prueba FK
pos = mc.get_coords() # leer posición real
mc.send_coords([x,y,z,rx,ry,rz], 30, 1) # verificar IK
angulos = mc.get_angles() # solución IK del robot
```

4.4 Rol 3: Ingeniero de Control

[CONTROL] Ingeniero de Control y Trayectorias

Problemas que lidera: P5

Tareas:

- P5: Definir las poses clave del ciclo: `init_pose`, `watch_pose`, `pick_pose`, `place_pose`.
- P5: Implementar el ciclo de agarre con `send_angles/send_coords + set_gripper_value`.
- Calibrar velocidades y tiempos de espera para movimientos seguros y repetibles.
- Ejecutar 5 ciclos consecutivos sin intervención humana y registrar la tasa de éxito.
- Usar `ik_solve(x,y,z)` del Rol de Cinemática para calcular los ángulos de agarre.
- Implementar manejo de errores de comunicación (reintentos, *timeouts*).

Entregables:

- `control.py` con funciones: `goto_pose()`, `pick()`, `place()`, `run_cycle()`.
- Log de 5 ciclos: tiempo de ciclo, éxito/fallo por fase y causa de fallo.
- Tabla de poses clave con ángulos y coordenadas cartesianas.
- Reporte de control (2 págs.): descripción del ciclo, métricas y análisis.

API requerida:

```
mc.send_angles(angles, speed) # mover a configuración
mc.send_coords(coords, speed, 1) # mover a posición
mc.set_gripper_value(100, 80) # gripper abierto
mc.set_gripper_value(0, 80) # gripper cerrado
mc.get_coords() # leer posición actual
```

4.5 Rol 4: Ingeniero de Visión

[VISIÓN] Ingeniero de Visión Computacional

Problemas que lidera: P6

Tareas:

- P6: Calibrar los rangos HSV para el objeto objetivo en las condiciones del laboratorio.
- P6: Detectar el contorno y calcular el centroide (c_x, c_y) en píxeles con OpenCV.
- P6: Implementar la transformación píxel $\rightarrow$ mm en el sistema de referencia del robot.
- Validar la precisión de detección para al menos 3 posiciones distintas del objeto.
- Manejar falsos positivos, iluminación variable y ausencia del objeto en la escena.
- Publicar la posición detectada para que el Líder de Integración la consuma en `main.py`.

Entregables:

- `vision.py` con función `detect_object(frame) -> (x_mm, y_mm) | None`.
- Tabla de validación: 3 posiciones reales vs. detectadas, con error en mm.
- Notebook de calibración HSV con imágenes de referencia.
- Reporte de visión (2 págs.): pipeline, rango HSV final y métricas de precisión.

API requerida:

```
import cv2
cap = cv2.VideoCapture(0)
ret, frame = cap.read()
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, lower_bound, upper_bound)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

5. Cronograma Semanal

Rol	Lunes	Martes	Miércoles	Jueves	Viernes	Fin de semana
Líder	Setup + Git	Arquitectura	Integración	Integración	Tests E2E	Demo + Defensa
Cinemática	Config API	FK + DH	IK + Evasión	Verificación	Test colisión	Ajuste + Defensa
Control	Config API	Poses base	Agarre prim.	Ciclo agarre	Tests reales	Ajuste + Defensa
Visión	Config cam.	HSV color	Centroide	Color+Robo	Integración	Vídeo + Defensa

6. Entregables

1. **Repositorio Git** con ramas por rol y al menos 3 *commits* por integrante.
2. **Código fuente**: `cinematica.py`, `control.py`, `vision.py`, `main.py`.
3. **Informe técnico grupal** (PDF, mínimo 10 páginas) con secciones por rol:
 - Sección Cinemática: DH, FK, IK, método de evasión de colisiones.
 - Sección Control: poses, ciclo de agarre, métricas.
 - Sección Visión: pipeline HSV, transformación píxel $\rightarrow$ mm, métricas.
 - Sección Integración: arquitectura, máquina de estados, log de sesión.
4. **Vídeo de demostración** (3–5 minutos): pipeline autónomo con al menos 5 ciclos.
5. **Presentación** (10–12 diapositivas) lista para defensa oral de 15 minutos.

7. Rúbrica de Evaluación

Criterio	Peso	Descripción
Cinemática (P1–P4)	35%	FK e IK correctas, tablas de error, evasión de colisiones justificada e implementada.
Control (P5)	20%	Ciclo de agarre funcional, 5 ciclos consecutivos, tasa de éxito $\geq 80\%$.
Visión (P6)	15%	Detección robusta, transformación píxel→mm, error < 15 mm.
Integración (P7)	20%	<code>main.py</code> autónomo, máquina de estados funcional, 5 ciclos completos.
Informe + Defensa	10%	Calidad del documento, claridad de la presentación, respuestas en defensa oral.

8. Referencia Rápida del API pymycobot

Método	Descripción
<code>MyCobot(port, baud)</code>	Inicializar conexión (<code>/dev/ttyUSB0</code> , 1000000).
<code>send_angles(angles, speed)</code>	Enviar vector de 6 ángulos en grados, velocidad 0–100.
<code>send_coords(coords, sp, m)</code>	Enviar posición $[x, y, z, rx, ry, rz]$ en mm/grados.
<code>get_angles()</code>	Leer ángulos actuales de los 6 <i>joints</i> (grados).
<code>get_coords()</code>	Leer posición actual del extremo $[x, y, z, rx, ry, rz]$.
<code>set_gripper_value(v, sp)</code>	Abrir/cerrar <i>gripper</i> : 0 = cerrado, 100 = abierto.
<code>get_gripper_value()</code>	Leer posición actual del <i>gripper</i> .
<code>is_controller_connected()</code>	Verificar conexión con el controlador.
<code>release_all_servos()</code>	Liberar todos los servos (modo libre).
<code>power_on()</code>	Activar motores del robot.

Inicialización estándar (todos los roles)

```
from pymycobot.mycobot import MyCobot
import time

mc = MyCobot('/dev/ttyUSB0', 1000000)
mc.power_on()
time.sleep(0.5)
assert mc.is_controller_connected(), "Error de conexión"

mc.send_angles([0, 0, 0, 0, 0, -45], 50) # pose de reposo segura
time.sleep(3)
print("Posición actual:", mc.get_coords())
```

9. Normas de Seguridad

1. **Nunca** dejar el robot operando sin supervisión humana.
2. **Siempre** comenzar en la pose `init_pose` antes de cualquier movimiento.
3. **Mantener** las manos fuera del espacio de trabajo durante la ejecución.
4. **Limitar** la velocidad a 30–50% durante las pruebas iniciales de cualquier trayectoria nueva.
5. **Implementar** un botón de parada de emergencia en el código (`Ctrl+C` con manejador de señal).
6. **Verificar** visualmente la trayectoria completa en modo lento antes de ejecutarla a velocidad normal.
7. **Reportar** cualquier comportamiento inesperado del robot al docente antes de continuar.